CAS CS 538. Solutions to Problem Set 7

Due electronically via Gradescope, Monday March 30, 2026 11:59pm

Problem 1. (25 points) Consider the following MAC (a variant of this was used for Wi-Fi encryption in 802.11b WEP). Let F be a PRF defined over $(\mathcal{K}, \mathcal{R}, \mathcal{X})$ where $\mathcal{X} := \{0,1\}^{32}$. Let CRC32 be any efficiently computable function mapping $m \in \{0,1\}^{\leqslant \ell}$ to a 32-bit string (in 802.11b, it was a simple and popular error-detecting code meant to detect random errors). Define the following MAC system (S, V):

$$S(k,m) := \left\{ r \xleftarrow{R} \mathcal{R}, t \twoheadleftarrow F(k,r) \oplus \text{CRC32}(m), \text{output } (r,t) \right\},$$
$$V(k,m,(r,t)) := \{\text{accept if } t = F(k,r) \oplus \text{CRC32}(m) \text{ and reject otherwise}\}.$$

Show that this MAC system is insecure.

Solution. The main issue with this MAC setup is primarily the way F is setup in this system. Basically, the MAC reveals info about the hidden values to the adversary. This can then be computed by the adversary to make up a new valid MAC.

Game

Let $\mathcal{A}$ be the efficent adv. attacking the MAC protocol. We are trying to make a game, such that the challenger outputs $\{V(k,\cdot,(\cdot,\cdot)) \to \text{ACCEPT}\}$ from $\mathcal{A}$'s input, when it shouldn't.

- Challenger gives $\mathcal{A}$ access to the following Oracles:
  - $S(k,\cdot)$
  - $V(k,\cdot,(\cdot,\cdot))$
- $\mathcal{A}$ chooses rnd $m_1 \in M$.
- $\mathcal{A}$ querys $S(k,m_1)$, recieves (r_1,t_1).
- $\mathcal{A}$ now chooses rnd $m_2 \in M \backslash m_1$.
- $\mathcal{A}$ computes $t_2 = (t_1 \oplus \text{CRC32}(m_1)) \oplus \text{CRC32}(m_2)$
- $\mathcal{A}$ queries $V(m_2,(r_1,t_2))$, gets {ACCEPT, REJECT}.

Analysis

I'm calculating $\text{CMAadv}[\mathcal{A}, \text{MAC}] = |\Pr[W_0] - \Pr[W_1]|$, where $Pr[W_b]$ is just the chance that the game will output ACCEPT in a particular game.

Game 0 (rnd) In this game, $S(k,\cdot)$ just outputs a rnd value $\in \{0,1\}^{32}$. So the chances V will actually work is just 2^{32}. This is negligible.

Game 1 Now we are analyzing the constructed game. Note what $\mathcal{A}$ recieves upon first query, (r_1, t_1). $\mathcal{A}$ knows that $t_1 = F(k, r_1) \oplus \text{CRC32}(m_1)$. So then, $F(k, r_1) = t_1 \oplus \text{CRC32}(m_1)$. Then it is possible to calculate:

$$\begin{aligned} t_2 &= (t_1 \oplus \text{CRC32}(m_1)) \oplus \text{CRC32}(m_2) \\ &= F(k, r_1) \oplus \text{CRC32}(m_2) \end{aligned}$$

Note how V operates. If $\mathcal{A}$ were to now query the challenger with $(m_2, (r_1, t_2))$, V is now checking if:

$$\begin{aligned} t_2 &= F(k, r_1) \oplus \text{CRC32}(m_2) \\ F(k, r_1) \oplus \text{CRC32}(m_2) &= F(k, r_1) \oplus \text{CRC32}(m_2) \end{aligned}$$

As both sides are exactly equal to one another, the challenger will always output ACCEPT in this case.

Analysis

Therefore, we get the following:

$$\begin{aligned} \text{CMAadv}[\mathcal{A}, \text{MAC}] &= |\Pr[W_0] - \Pr[W_1]| \\ &= |\frac{1}{2^{32}} - 1| \\ &\approx 1 \end{aligned}$$

This is not negligible. Therefore this proves that the MAC protocol is insecure. $\square$

Problem 2. (30 points) We want to build a MAC system $\mathcal{I}$ using two MAC system $\mathcal{I}_1 = (S_1, V_1)$ and $\mathcal{I}_2 = (S_2, V_2)$, so that if at some time one of $\mathcal{I}_1$ or $\mathcal{I}_2$ is broken (but not both) then $\mathcal{I}$ is still secure. Put another way, we want to construct $\mathcal{I}$ from $\mathcal{I}_1$ and $\mathcal{I}_2$ such that $\mathcal{I}$ is secure if either $\mathcal{I}_1$ or $\mathcal{I}_2$ is secure. This is also called a *MAC combiner*.

For this problem, suppose that $\mathcal{I}_1$ and $\mathcal{I}_2$ are deterministic MAC systems (see the definition on textbook page 217 in Section 6.1), and that both have tag space $\{0, 1\}^n$. Define the deterministic MAC system $\mathcal{I} = (S, V)$, where

$$S((k_1, k_2), m) := S_1(k_1, m) \oplus S_2(k_2, m).$$

Show that $\mathcal{I}$ is a MAC combiner of $\mathcal{I}_1$ and $\mathcal{I}_2$.

Solution. This can be proved through a reduction. Firstly, note the construction of $\mathcal{I}$:

$$S_1(k_1, m) \oplus S_2(k_2, m) = \mathcal{I}_1 \oplus \mathcal{I}_2$$

By logic and the laws of XOR, we are essentially just proving $(\mathcal{I}_1 \to \mathcal{I}) \vee (\mathcal{I}_2 \to \mathcal{I})$. So by proving that $\mathcal{I}_1$ is secure, we prove $\mathcal{I}$ is secure. The proof for $\mathcal{I}_2$ will be the same thing.

Game

Let $\mathcal{A}$ be the efficent adversary attacking $\mathcal{I}$, assumed to have some adv ϵ. Then, let $\mathcal{B}$ be another efficent adversary, attacking $\mathcal{I}_1$. Assume the contrapositive: If $\mathcal{I}$ is insecure, then $\mathcal{I}_1$ is insecure.

- $\mathcal{B}$ can query $S_1(k_1, \cdot)$. $\mathcal{A}$'s query will be intercepted by $\mathcal{B}$.
- Before anything, $\mathcal{B}$ will choose rnd $k_2 \to \mathcal{K}_2$.
  - $\mathcal{B}$ is now essentially simulating $\mathcal{I}_2$. This is possible, as we assume that we know how $\mathcal{I}_2$ works.
- Whenever $\mathcal{A}$ makes a query with m:
  - $\mathcal{B}$ forwards query to Challenger, gets $t_1 = S_1(k_1, m)$.
  - $\mathcal{B}$ then computes $t_2 = S_2(k_2, m)$.
  - $\mathcal{B}$ computes $t = t_1 \oplus t_2$.
  - $\mathcal{B}$ sends t to $\mathcal{A}$
- $\mathcal{A}$ will output some forgery (m^*, t^*). $\mathcal{B}$ intercepts this.
- $\mathcal{B}$ then computes:
  - $t_2^* = S_2(k_2, m^*)$
  - $t_1^* = t^* \oplus t_2^*$
- $\mathcal{B}$ returns forgery (m^*, t_1^*).

Analysis

Validity If $\mathcal{A}$ wins, by initial assumption, then (m^*, t^*) is a valid forged tag. By the definition of $\mathcal{I}$:

$$t^* = S_1(k_1, m^*) \oplus S_2(k_2, m^*)$$

Therefore what $\mathcal{B}$ computes is then:

$$\begin{aligned} t_1^* &= t^* \oplus S_2(k_2, m^*) \\ &= (S_1(k_1, m^*) \oplus S_2(k_2, m^*)) \oplus S_2(k_2, m^*) \\ &= S_1(k_1, m^*) \end{aligned}$$

By the initial assumption, this means that $\mathcal{B}$'s forgery will also then be valid.

Freshness If $\mathcal{A}$ wins, m^* is never sent to $\mathcal{I}$. $\mathcal{B}$ always forwards $\mathcal{A}$'s queries to $\mathcal{I}_1$. Therefore, if $\mathcal{A}$ has won, then $\mathcal{B}$ hasn't sent any new message m^* to $\mathcal{I}_1$. Therefore $\mathcal{B}$'s forgery is a valid existential tag for $\mathcal{I}_1$.

End

Since, when $\mathcal{A}$ wins, $\mathcal{B}$ also wins, the advantage of $\mathcal{B}$ will be at least that of $\mathcal{A}$:

$$\mathsf{MACadv}[\mathcal{B}, \mathcal{I}_1] \geqslant \mathsf{MACadv}[\mathcal{A}, \mathcal{I}]$$

By our initial assumption, then $\mathcal{B}$ will therefore then be non-negligible. This proves the contrapositive, therefore proving the OG statement. $\square$

Problem 3. (45 points) For this problem, it may be helpful to review discussion 7 and section 9.4.2 in the textbook.

Consider the following candidate authenticated encryption scheme. The idea is to apply a MAC to a plaintext, add a checksum value to detect decryption errors[1], and then encrypt the plaintext, MAC, and checksum value together.

Let $\mathcal{E} = (E, D)$ be a block cipher with $\mathcal{M} = \mathcal{K} = \mathcal{C} = \{0,1\}^{128}$, and $\mathcal{I} = (S, V)$ be a MAC with tag space $\{0,1\}^{128}$. For the checksum, we'll use a function $\mathsf{Checksum} : \{0,1\}^* \to \{0,1\}^8$ that outputs a one-byte tag.

Define the scheme $\mathcal{E}' = (E', D')$ as follows:

$E'((k_e, k_m), m)$:

$t \twoheadleftarrow S(k_m, m)$
$x \twoheadleftarrow \mathsf{Checksum}(m \| t)$
$c \twoheadleftarrow$ encryption of $m \| t \| x \| 0^{120}$ under k_e,
  using E in randomized CBC mode
Return c

$D'((k_e, k_m), c)$

$(m \| t \| x \| 0^{120}) \twoheadleftarrow$ decryption of c under k_e,
  using D in randomized CBC mode
If $x \neq \mathsf{Checksum}(m \| t)$:
  Output reject
Elif $V(k_m, m, t) = \mathsf{reject}$:
  Output reject
Else:
  Output m

Notice that E' pads the plaintext with 120 bits of zeros. You may assume that all message sizes are multiples of 128 bits. This ensures that the second-to-last block of every ciphertext is an encryption of the tag, and the last block is an encryption of the checksum followed by 120 zeros.

Prove that $\mathcal{E}'$ is not AE-secure.

Solution. Your solution goes here

[1] Of course, verifying the MAC also suffices to detect decryption errors. Adding the checksum is an attempt at redundancy.